

Documento del Proyecto Final: Implementación DevSecOps

Integración de prácticas DevSecOps en la aplicación Puyo Express

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Grupo 6

Integrantes

Moises Benalcazar
David Cepeda
Mesias Mariscal
Carlos Ñato
Denise Rea

Fecha de entrega: 21 de Julio de 2026

Índice

1. Introducción y Contexto de la Aplicación	1
1.1. Descripción del aplicativo	1
1.2. Objetivo de seguridad	1
2. Arquitectura e Inventario de Activos	1
2.1. Diagrama de Arquitectura	1
2.2. Tabla de Activos	2
3. Seguridad en Autenticación y Prevención de Ataques	3
3.1. Protección de Contraseñas (Hashing)	3
3.2. Mitigación de Ataques de Fuerza Bruta	3
4. Trazabilidad, Logs y Observabilidad	3
4.1. Estructura de los Logs	3
4.2. Integración con el Inventario de Activos	3
4.3. Preparación para Herramientas de Observabilidad	4
5. Hardening	4
5.1. Hardening en el Código	4
5.2. Hardening del Servidor	4
5.3. Ocultamiento de Versiones y Fingerprinting	5
6. Conclusiones y Trabajo Futuro	5
6.1. Conclusiones de Implementación	5
6.2. Mejoras a futuro	5

Sección 1

Introducción y Contexto de la Aplicación

1.1 Descripción del aplicativo

Puyo Express es una plataforma integral de gestión de entregas y logística basada en una arquitectura cliente-servidor moderna. Su funcionamiento central gira en torno a conectar restaurantes, clientes y repartidores, permitiendo el registro de usuarios, creación de órdenes de comida o encomiendas y el seguimiento continuo del estado de los envíos. Está desarrollada utilizando el framework Spring Boot para exponer una API RESTful robusta en el backend, mientras que el cliente consume dichos servicios a través de un frontend responsivo y dinámico (React/Vite). Todos los datos transaccionales, perfiles de usuario e históricos de pedidos son almacenados en una base de datos relacional PostgreSQL. Para potenciar la seguridad, modularidad y portabilidad, todo este ecosistema se despliega de manera orquestada y aislada en redes privadas mediante contenedores de Docker.

1.2 Objetivo de seguridad

El propósito de incorporar el ciclo DevSecOps en esta fase final es garantizar que la información sensible de los usuarios esté protegida desde la persistencia (cifrado) hasta el acceso mediante autenticación robusta y asegurar que todas las acciones relevantes sean trazables mediante auditoría, reduciendo así la superficie de ataque y los riesgos asociados a filtración de información (Data Breach).

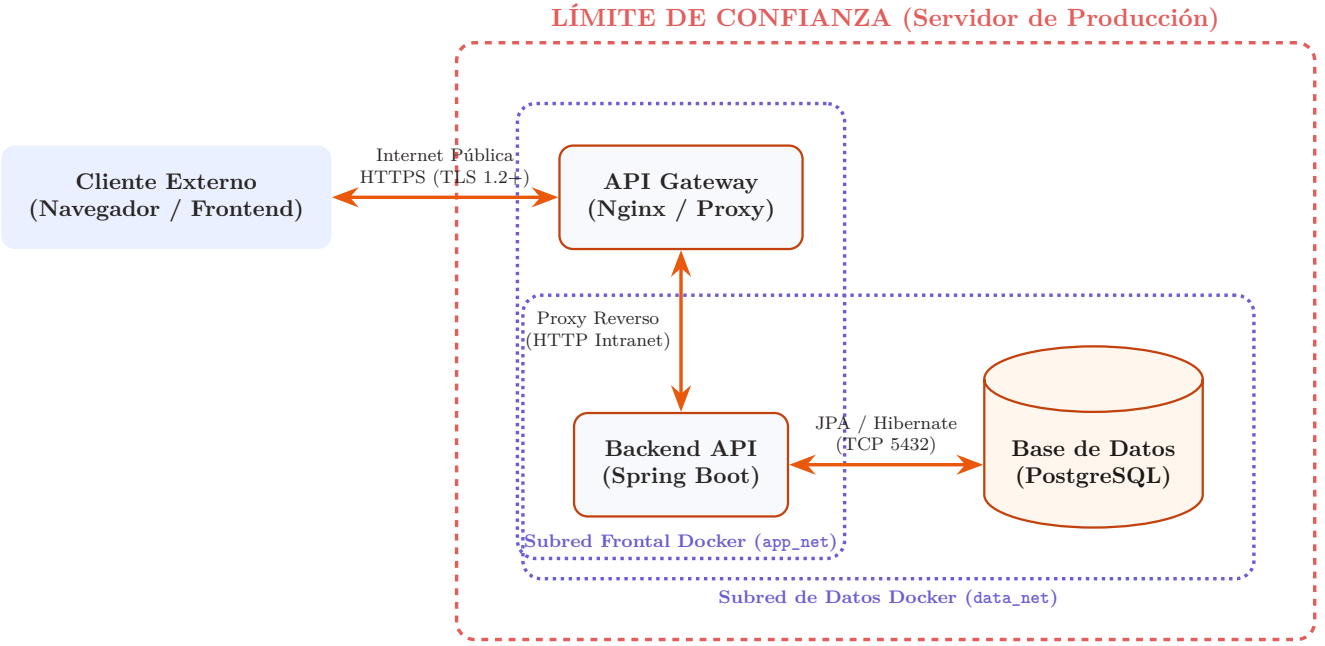
Sección 2

Arquitectura e Inventario de Activos

2.1 Diagrama de Arquitectura

Límites de Confianza

Los servicios internos (Base de datos PostgreSQL y backend Spring Boot) se aíslan en las subredes `data_net` y `app_net`. Únicamente el proxy inverso (Frontend/Nginx) mapea un puerto al host público.



2.2 Tabla de Activos

A continuación, se listan los principales activos de información del sistema según lo establecido en el Inventario de Activos.

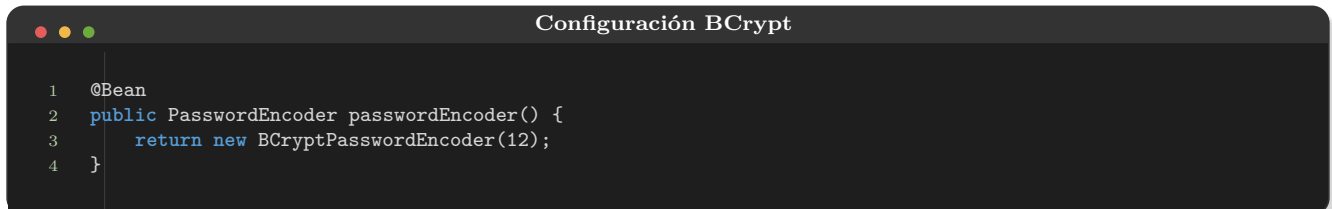
Activo	Ubicación y Endpoints	Criticidad	Protección
Contraseñas de Usuarios	DB (users.password) /api/auth/*	Crítico	Hash BCrypt (costo 12)
Teléfono y Dirección	DB (orders.customer_*) /api/orders/*	Confidencial	AES-256-GCM por campo
Correo electrónico	DB (users.email) /api/auth/*, /api/users/*	Confidencial	Cifrado de volumen y TLS
Claves Secretas (JWT, AES)	Docker secrets Interno en backend	Crítico	256 bits aleatorios, sin Git
Tokens JWT (Sesión)	Cookies del Cliente /api/*	Crítico	HttpOnly, SameSite, Secure
Credenciales DB	Docker secrets Interno	Crítico	Gestión manual fuera de Git
Logs de Auditoría	stdout del contenedor /api/*	Confidencial	Sanitación (Sin payloads ni PII)

Sección 3

Seguridad en Autenticación y Prevención de Ataques

3.1 Protección de Contraseñas (Hashing)

Las contraseñas de los usuarios se almacenan utilizando el algoritmo de derivación de claves **BCrypt** con un factor de trabajo adecuado (costo 12). Este mecanismo previene ataques de fuerza bruta y uso de tablas arcoíris mediante salting automático.



```
Configuración BCrypt

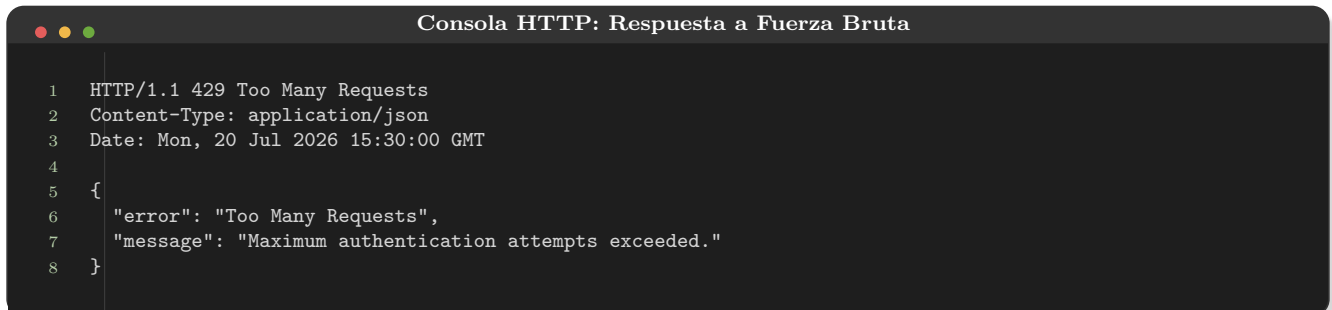
1  @Bean
2  public PasswordEncoder passwordEncoder() {
3      return new BCryptPasswordEncoder(12);
4  }
```

3.2 Mitigación de Ataques de Fuerza Bruta

Se implementa un mecanismo para penalizar los intentos erróneos de inicio de sesión.

- Lógica de bloqueo: Se utiliza un filtro especializado (**AuthRateLimitFilter**) configurado para restringir las peticiones a los endpoints de autenticación. Se permite un máximo de 10 intentos por cada ventana de 15 minutos, y en caso de superar este límite, se bloquean las solicitudes temporalmente para proteger el acceso a las cuentas de los usuarios.

Evidencia de Bloqueo:



```
Consola HTTP: Respuesta a Fuerza Bruta

1  HTTP/1.1 429 Too Many Requests
2  Content-Type: application/json
3  Date: Mon, 20 Jul 2026 15:30:00 GMT
4
5  {
6      "error": "Too Many Requests",
7      "message": "Maximum authentication attempts exceeded."
8  }
```

Sección 4

Trazabilidad, Logs y Observabilidad

4.1 Estructura de los Logs

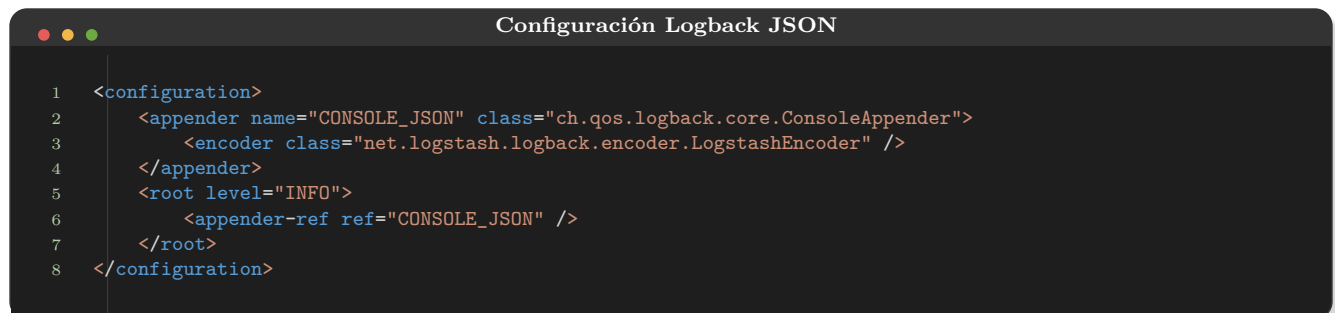
El sistema registra de forma estructurada los eventos relevantes de auditoría y seguridad sin incluir cargas útiles (payloads), contraseñas ni datos PII (teléfonos o direcciones).

4.2 Integración con el Inventario de Activos

Los logs se asocian con los accesos e interacciones hacia los activos confidenciales enumerados en el inventario, permitiendo detectar de forma temprana intentos de acceso no autorizados o comportamientos anómalos.

4.3 Preparación para Herramientas de Observabilidad

El backend se encuentra configurado con la dependencia `logstash-logback-encoder` y un archivo `logback-spring.xml` que unifica la salida de la consola.



```
1 <configuration>
2   <appender name="CONSOLE_JSON" class="ch.qos.logback.core.ConsoleAppender">
3     <encoder class="net.logstash.logback.encoder.LogstashEncoder" />
4   </appender>
5   <root level="INFO">
6     <appender-ref ref="CONSOLE_JSON" />
7   </root>
8 </configuration>
```

Esto asegura que la salida esté siempre formateada como un JSON válido por línea, permitiendo que herramientas SIEM y plataformas como Elasticsearch, Splunk o Datadog puedan ingerir e indexar cada campo de auditoría automáticamente sin depender de configuraciones engorrosas de parseo de texto.

Evidencia de Log JSON:



```
1 {"@timestamp":"2026-07-20T15:35:12.456Z", "message":"Authentication failed for user: admin", "logger_name":"AuthRateLimitFilter", "level":"WARN", "client_ip":"192.168.1.45", "event_type":"AUTH_FAILURE"}
```

Sección 5

Hardening

5.1 Hardening en el Código

El cifrado a nivel de columna (AES-256-GCM) para teléfonos y direcciones garantiza la confidencialidad de los datos sensibles en reposo, protegiéndolos de exposiciones si ocurriera un volcado de la base de datos, lo cual es una buena práctica crucial para mitigar violaciones de privacidad y cumplir los lineamientos del inventario de activos. Adicionalmente, el uso de Data Transfer Objects (DTOs) con validación estricta de entradas previene inyecciones y asignaciones masivas no autorizadas.

Por otro lado, se ha integrado un manejador global de excepciones mediante la clase `ApiExceptionHandler.java`. Esta es una buena práctica fundamental para ocultar los detalles internos de implementación. Garantiza que ninguna excepción de sistema inesperada logre retornar un *stacktrace* o errores crudos de la base de datos al cliente. En su lugar, intercepta el fallo localmente y devuelve una respuesta HTTP 500 estándar y limpia, cerrando la brecha de Information Disclosure.

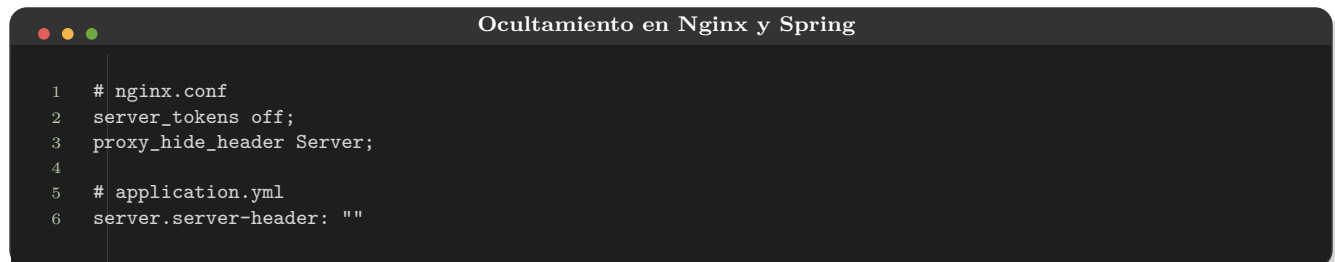
5.2 Hardening del Servidor

Se establecen límites estrictos en la recepción de payloads HTTP para mitigar ataques DoS a través del filtro `RequestSizeLimitFilter`. El entorno del host valida cortafuegos y el backend restringe puertos mediante redes Bridge aisladas en Docker. A nivel de frontend/proxy, un API Gateway desarrollado en Go inyecta de forma proactiva cabeceras de seguridad estrictas como Content Security Policy (CSP), X-Frame-Options y Permissions-Policy.

5.3 Ocultamiento de Versiones y Fingerprinting

Ocultar las cabeceras HTTP que revelan información interna impide que atacantes utilicen herramientas de reconocimiento (Fingerprinting) para mapear el software, identificar sus versiones y lanzar exploits específicos. En el proxy reverso, la directiva `server_tokens off;` evita que Nginx exponga su número de versión, mientras que `proxy_hide_header Server;` bloquea la propagación de la identidad del servidor de origen hacia el cliente.

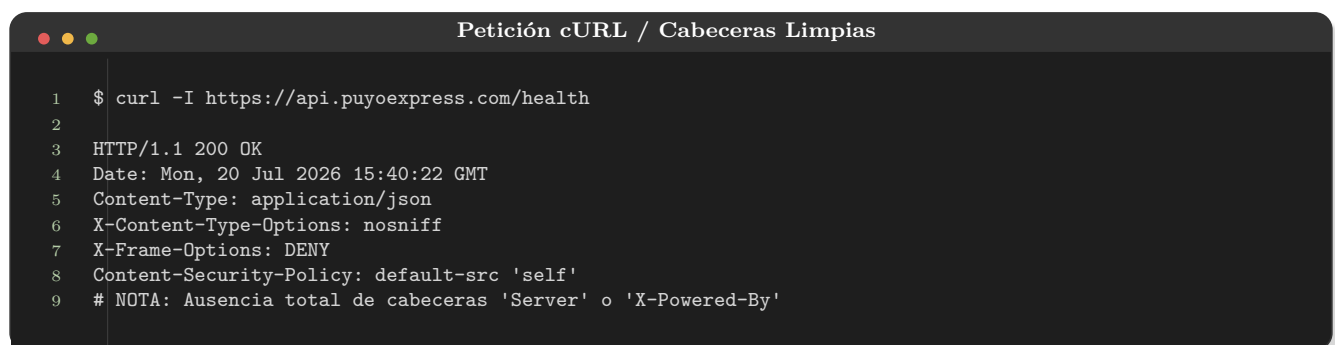
En el backend, se configuró la propiedad `server.server-header:` dentro de `application.yml` para suprimir la cabecera predeterminada de `.Apache Coyote.o "Tomcat.en` todas las peticiones HTTP. Combinando esto con el manejador de errores `ApiExceptionHandler.java` que anula los *stacktraces*, se mitiga efectivamente cualquier fuga de detalles internos tecnológicos que un atacante pudiera aprovechar.



```
Ocultamiento en Nginx y Spring

1 # nginx.conf
2 server_tokens off;
3 proxy_hide_header Server;
4
5 # application.yml
6 server.server-header: ""
```

Evidencia de Cabeceras HTTP Seguras (Fingerprinting mitigado):



```
Petición cURL / Cabeceras Limpias

1 $ curl -I https://api.puyoexpress.com/health
2
3 HTTP/1.1 200 OK
4 Date: Mon, 20 Jul 2026 15:40:22 GMT
5 Content-Type: application/json
6 X-Content-Type-Options: nosniff
7 X-Frame-Options: DENY
8 Content-Security-Policy: default-src 'self'
9 # NOTA: Ausencia total de cabeceras 'Server' o 'X-Powered-By'
```

Sección 6

Conclusiones y Trabajo Futuro

6.1 Conclusiones de Implementación

Durante la ejecución de las auditorías e implementaciones técnicas, el principal desafío fue comprender el impacto de la fuga de secretos persistentes en el historial de Git (como claves de bases de datos y cifrado), requiriendo una estricta rotación y reestructuración del control de versiones. Asimismo, el proceso de refactorizar la lógica para evitar problemas de autorización insegura (IDOR/BOLA), asegurando que cada usuario interactúe solo con sus propios datos, demostró la vital importancia de utilizar DTOs validados para sanear la capa de persistencia en Spring Boot.

6.2 Mejoras a futuro

Para continuar madurando el ciclo DevSecOps del proyecto en iteraciones futuras, se recomienda:

- **Gestión automatizada del código estático:** Integrar herramientas SAST (Static Application Security Testing) directamente en un pipeline CI/CD para bloquear vulnerabilidades en tiempo de compilación.

- **Mitigación avanzada:** Implementar una protección completa contra ataques CSRF (Cross-Site Request Forgery) exigiendo tokens robustos, y forzar la bandera **Secure** por defecto en las cookies críticas de la aplicación.
- **Evolución y desempeño segura:** Sustituir la estrategia de persistencia actual (`ddl-auto`) por migraciones controladas usando herramientas como Flyway o Liquibase. Asimismo, emplear técnicas de paginación para mitigar riesgos de denegación de servicio (DoS) ante consultas masivas de información.